Design and Performance Evaluation of a Minimal Non-Blocking HTTP/1.1 Server

Arthur Herbette

April 11, 2026

Abstract

This report presents the design and performance evaluation of a minimal HTTP/1.1 server implemented in C. The server relies on non-blocking I/O and a multithreaded worker architecture using `SO_REUSEPORT` to handle concurrent connections efficiently. The event loop was first built on `epoll` with `sendfile(2)` for the body, then fully re-wired on `io_uring` (accept/recv/send plus `IORING_OP_SEND` of an mmap'd file cache for the body). On a 100 KiB static file, measured back-to-back in the same session: the `epoll` build sustains ≈ 287 k req/s, within noise of `nginx` on this hardware while the `io_uring` build sustains ≈ 165 k req/s. The gap is almost entirely the cost of giving up `sendfile`'s zero-copy for a kernel memcpy out of the mmap region. In exchange, the `io_uring` build cuts the tail from $p99 \approx 40$ ms to $p99 \approx 7.6$ ms, bringing it within ≈ 2.5 ms of `nginx`.

1 Introduction

The goal of this project was to gain a deeper understanding of HTTP and network programming. You can find the GitHub repository at this link. I started this project as a simple blocking, single-threaded, loop-based server. After understanding those fundamentals I implemented header parsing which enabled persistent (keep-alive) connections. The goal after that introduction was to have a very efficient and scalable server. Those reasons motivated the transition to a non-blocking server architecture, allowing the server to handle multiple connections concurrently using asynchronous I/O, and eventually to a fully multithreaded design.

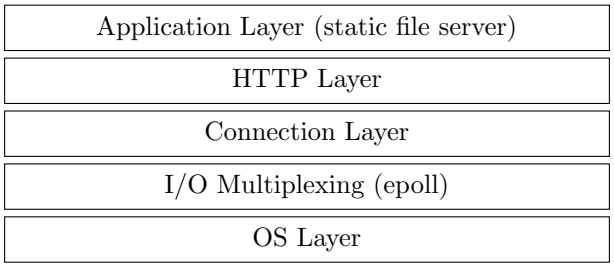
2 Architecture

The server follows an event-driven architecture. It started as a single-threaded reactor pattern and evolved into a multi-threaded design where each worker thread owns its own event loop and its own listen socket via `SO_REUSEPORT`. The event loop was initially built on `epoll` and was subsequently rewritten on top of `io_uring` (see Section 2.2). The architecture is divided into 7 layers:

- **core**: coordinates the server and orchestrates interactions between components
- **io**: owns the `io_uring` ring and submits/completes I/O operations (replaces the earlier epoll layer)
- **HTTP**: parses requests, turns plain text into `http_request_t`
- **net**: handles connection setup, TCP listen and accept
- **os**: provides low-level read/write operations and filesystem access
- **static**: handles all transactions which use static files (i.e., `index.html`)
- **util**: contains all utilities of the server; at the moment it contains only `buffer_t` which serves to store and consume all data from the connection

The system is divided into layers so that each component exposes a clear abstraction and minimises coupling with the others.

To better understand this server we can first explain what each layer does in the stack:



Incoming data flows upward through the stack (OS → HTTP), while responses are generated at the application layer and propagated downward to the socket.

OS Layer The OS layer's responsibilities are: sockets, file descriptors, read/write/accept, and filesystem access.

Event Layer The event layer (epoll) turns blocking OS events into asynchronous events. The question it tries to answer is: *which fd is ready?*

Connection Layer The connection layer abstracts over raw sockets; its responsibilities are: input/output buffers, write offsets, connection state, and the keep-alive lifecycle.

HTTP Layer It only works with plain text. The goal is to: parse requests, detect headers, handle keep-alive, and build responses. It transforms raw bytes into a structured request.

Application Layer (static) This is the highest-level logic. At the current moment, as it serves only GET, it only serves files, but could later become: API endpoints, dynamic content, routing.

2.1 Multithreaded Worker Model

The final architecture uses N worker threads (currently 8), each running its own epoll loop. Rather than having a single main thread accept connections and distribute them to workers via a queue, which introduces cross-thread synchronisation on every connection; each worker binds its own copy of the listen socket using `SO_REUSEPORT`. The kernel itself distributes incoming connections across workers with no mutex, no `eventfd` wakeup, and no shared queue. Each worker is entirely self-contained:

- its own `epoll` instance
- its own listen socket (`SO_REUSEPORT`)
- its own connection pool (`conn_pool_t`)

This design eliminates the cross-thread dispatch overhead that was previously responsible for roughly 20 % of latency under load. Compared to the earlier dispatch model, the hot path per connection drops from: `lock` → `memcpy` → `eventfd write` → `wakeup` → `lock` → `drain` → `epoll_ctl` to simply: `epoll fires` → `accept4` → `epoll_ctl`.

2.2 Migration from epoll to io_uring

After the `epoll` implementation reached parity with `nginx`, `perf` profiles showed that roughly a third of CPU time was being spent in individual syscalls (`epoll_ctl`, `read`, `write`, `sendfile`). To reduce this overhead, the event loop was rewritten on top of `io_uring`: each worker now owns a submission/completion ring instead of an `epoll` instance, and I/O is driven by submitting SQEs rather than reacting to readiness notifications.

The reactive readiness model (*the fd is ready; now call read*) was replaced with a proactive completion model (*read this; tell me when it's done*). Concretely:

- **accept**: a single `IORING_OP_ACCEPT` in multishot mode (`io_uring_prep_multishot_accept`) is submitted once per worker and delivers every new connection as a CQE until it is cancelled.
- **recv/send (headers)**: each connection owns an `inflight` counter; `io_uring_prep_recv` and `io_uring_prep_send` are queued with the connection pointer packed with the operation type in the SQE user-data (the low 3 bits of the aligned pointer encode the `io_op_t`).
- **file body**: the static cache `mmap`s each file at startup; the send path is an `IORING_OP_SEND` of the mmap'd region (Section 2.2.1). The earlier `sendfile(2)` call is gone.
- **worker loop**: a single `io_uring_submit_and_wait` call per iteration both submits all queued SQEs and blocks for at least one completion; CQEs are then drained in a tight `io_uring_for_each_cqe` loop.

Each worker thread is still entirely self-contained: it has its own ring, its own listen socket, its own connection pool. Because the ring already acts as a wait point, the earlier `eventfd` and `epoll_ctl` machinery is gone from the hot path.

2.2.1 Choosing a body-transfer primitive

The natural `io_uring` replacement for `sendfile(2)` is a pair of linked `IORING_OP_SPLICE` SQEs: file → pipe, pipe → socket, chained via `IOSQE_IO_LINK`. We implemented this first and ran into two kernel-level obstacles on the test machine (Linux 7.1.3):

1. **Linked SQEs cancel on short completions.** `IOSQE_IO_LINK` runs the next SQE only if the previous one succeeds *fully*. A short result on the first splice (common when the pipe fills) cancels the second with `-ECANCELED`. The documentation for a few other opcodes (`read`/`write`/`recv`/`send`) carves out an exception for short results; splice is not in that list.
2. `pipe-user-pages-soft` **silently caps per-call splice size.** With one pipe per pooled connection ($\approx$ 4096 pipes), the user's total pinned pipe pages exceeds `/proc/sys/fs/pipe-user-pages-soft` (16384 pages = 64 MiB). `F_SETPIPE_SZ` reports success and `F_GETPIPE_SZ` still returns the requested size, but `io_uring`'s `splice` actually caps per call to 2 pages (8 KiB). A 100 KiB body then takes thirteen async-worker dispatches; measured end-to-end this gave $\approx$ 10 k req/s with a uniform $\approx$ 41 ms latency floor.

We therefore abandoned splice and chose a simpler primitive: `mmap` every cached file at startup, then push the body with a single `IORING_OP_SEND` of the mmap'd region. This gives up zero-copy — the kernel has to copy from the mapped page cache into the socket buffer — but a 100 KiB copy at $\approx$ 15 GB/s costs $\sim 7\,\mu$s and compares well against losing an entire async-worker dispatch per 8 KiB. We also tried `IORING_OP_SEND_ZC` to recover zero-copy, but on this kernel, under keep-alive load, we observed millions of `wrk` read errors; the interaction between the `F_NOTIF` completion and immediate connection reuse needs more investigation before that path can be revived.

Trade-offs The move to `io_uring` is a strict reduction in syscalls but is not free. The connection state machine becomes a completion state machine: operations can complete partially (`res < len`) and have to be re-submitted, close has to wait on `inflight` reaching zero so completions do not fire on a reused slot, and errors surface as negative `cqe->res` rather than `errno`. Each of these edge cases was a separate class of bug during the rewrite.

3 Implementation Challenges

The primary challenges were related to incremental I/O and state management. Unlike blocking implementations, requests may arrive in partial segments, requiring persistent buffering and repeated parsing attempts. Handling keep-alive connections introduced additional complexity, since multiple requests can coexist within the same input buffer.

A subtler challenge was the interaction between edge-triggered epoll (`EPOLLET`) and partial reads. With level-triggered epoll the kernel re-notifies as long as data is available, which is forgiving of incomplete reads. With `EPOLLET` the notification fires exactly once on the state transition; if the entire available data is not drained in that single dispatch, the connection silently stalls. This required careful loop structures in `handle_read` to always drain until `EAGAIN`.

Another challenge specific to the multithreaded design was ensuring correct use of `epoll_ctl`. The `EPOLL_CTL_MOD` operation requires the file descriptor to already be registered via `EPOLL_CTL_ADD`; calling `MOD` on an unregistered fd returns `ENOENT` silently, leaving the connection permanently stalled. Systematic error checking on all `epoll_ctl` calls was necessary to surface these issues during development.

4 Experimental Setup

Benchmarks were performed using `wrk`:

```
wrk -t8 -c400 -d30s --latency http://127.0.0.1:8080/<file>
```

Each server was warmed up for 10 s and then measured over three independent 30 s runs. To avoid cross-session drift, the `io_uring` build, the `epoll` build, and `nginx` were all benchmarked back-to-back within a single session.

The setup is a ThinkPad T16 with an Intel Core Ultra 7 155U (14 cores) @ 4.80 GHz with 32 GiB of RAM running Arch Linux. `nginx` runs on the same machine on port 8081 with 8 workers and `SO_REUSEPORT`, serving the identical file tree. All benchmarks share the same hardware with the load generator: `wrk` and the server under test compete for the same 14 cores, so the numbers below are not the server's ceiling on dedicated hardware, they are a self-consistent comparison on one machine.

5 Discussion

5.1 Effect of Keep-Alive

The transition to a non-blocking architecture significantly increased throughput, especially when combined with persistent connections. Without keep-alive, one request corresponds to exactly one TCP connection. This means that for every request the server must: `accept()`, TCP handshake, read the request, write the response, close socket. With keep-alive, one connection can carry many requests. The connection stays open, so we avoid repeating the accept and close process, which reduces syscall count and kernel allocations.

Measured impact of keep-alive (single-threaded baseline, `ab -n 10000 -c 100`):

Mode	Req/s	Notes
Without keep-alive	51,105	full TCP handshake per request
With keep-alive	152,935	connections reused across requests

Adding keep-alive does not especially reduce the latency of each individual request but adds significant throughput and scalability by removing the TCP setup cost.

5.2 Optimisation Journey

The server was profiled iteratively using `perf record` and `perf report` after each change. The following optimisations were applied in order of impact:

1. **SO_REUSEPORT per-worker sockets**: eliminated the centralised accept-and-dispatch model with `eventfd` wakeups. Each worker binds its own listen socket; the kernel distributes connections with no cross-thread synchronisation.

2. **Keep-alive typo fix**: a typo in the HTTP version comparison (`HTTTP/1.1` instead of `HTTP/1.1`) caused every connection to be treated as HTTP/1.0 and closed after each response. Fixing this alone approximately doubled request throughput by enabling connection reuse.

3. **Parser cleanup**: replaced `strncpy`-based header copying with bounded `memcpy` using lengths already computed during scanning. Eliminated in-place buffer mutation (writing `\0` into the buffer and restoring) which was cache-unfriendly and fragile.

4. **Eliminated `dup()` per request**: the original code called `dup()` on the cached file descriptor so that `sendfile` could have its own file offset. Since `sendfile` accepts an explicit `offset` pointer, the `dup()` is unnecessary; removing it eliminated a syscall and a `close()` on every request.

5. **TCP_CORK gating**: `TCP_CORK` was being set and unset on every request regardless of file size, adding two `setsockopt` syscalls per response. Gating behind a file-size threshold (only cork when file > 16 KiB) removed this overhead for the common case of small files.

6. **io_uring event loop (complete)**: replaced `epoll` + `read`/`write` + `sendfile(2)` with a per-worker `io_uring` ring: multishot `accept`, `IORING_OP_RECV`/`SEND` for header I/O, and `IORING_OP_SEND` of an `mmap`'d static-file cache for the body. Linked `IORING_OP_SPLICE` was tried first and discarded for the reasons in Section 2.2.1. Same-session measurements against the `epoll` build show a $\approx 40\,\%$ throughput regression and a $5\times$ smaller $p99$ tail; see Section 6.

5.3 Limitations

Performance behaviour can be interpreted through three regimes: connection-bound, CPU-bound, and network-bound execution. Enabling persistent connections moved the server from a connection-bound regime to a CPU-bound regime, significantly increasing throughput. The multithreaded `SO_REUSEPORT` design then distributed that CPU work across cores.

Under the `epoll` architecture, `perf` attributed $\sim 11\,\%$ of cycles to HTTP parsing, $\sim 10\,\%$ to `epoll_ctl`, $\sim 10\,\%$ each to `read` and `write`, and $\sim 7\,\%$ to `sendfile` — i.e., roughly $38\,\%$ of the budget sat in per-syscall mechanics. The `io_uring` rewrite was aimed at that budget: a single `io_uring_submit_and_wait` per loop iteration replaces one `epoll_wait` plus one syscall per I/O. See Section 6 for how far the rewrite actually got.

6 Benchmark Results

Same-session comparison — 100 KiB file The three servers were benchmarked back-to-back in a single session (three 30 s runs each, 8 threads, 400 concurrent connections, 10 s warm-up). Table 1 reports the median run.

Server	Req/s	p50	p75	p90	p99
This server (`io_uring`, `mmap+SEND`)	165,190	1.40 ms	2.75 ms	4.31 ms	7.55 ms
This server (`epoll`, `sendfile`)	287,722	715 μs	1.83 ms	21.52 ms	40.37 ms
`nginx` (epoll, `sendfile`)	275,595	0.99 ms	1.74 ms	2.63 ms	4.49 ms

Table 1: Static file throughput and latency percentiles, 100 KiB file, `wrk -t8 -c400 -d30s --latency`, median of three 30 s runs after a 10 s warm-up, all three servers run back-to-back in one session on the same hardware contended by `wrk`.

Throughput The `epoll` build is within noise of `nginx` on throughput; both use `sendfile(2)` and get the kernel to DMA directly from the page cache to the socket. The `io_uring` build is $\approx 40\,\%$ slower. The reason is simple: by moving the body to `IORING_OP_SEND` of an `mmap`'d region, the kernel has to copy from the mapped pages into the socket buffer. For 100 KiB at ~ 165 k req/s, that is ≈ 15 GB/s of memcpy sitting squarely in the hot path.

Latency tail Giving up `sendfile` cost throughput but mostly removed the fairness tail that the `epoll` build had: $p90$ drops from 21.5 ms to 4.3 ms, $p99$ from 40 ms to 7.6 ms. We did not instrument this directly, so the mechanism is still a hypothesis: in the `epoll` build, `handle_send` calls `sendfile(2)` in a tight loop until `EAGAIN`, which can monopolise a worker thread for the duration of a large file transfer and starve other connections scheduled to the same worker. The `io_uring` build submits a single SEND and returns to the drain loop, so the worker is free to interleave work across connections.

Where cycles go now A `perf record` run on the `io_uring` server during a full-load `wrk` is shown in Table 2. `sendfile64`, which was 9.4 % of samples in the previous measurement, is now absent — the port is complete. HTTP parsing is the single largest user-space hotspot at 16.8 %; the next largest resolved symbol is `handle_recv` at 9.5 %. A 9.8 % chunk sits at an unresolved libc address; the stack above it leads through `io_add_send` and `handle_send`, which is consistent with the kernel's socket-write memcpy path that this build now owns.

Symbol (user space)	Self
`http_parse_request`	16.8 %
unresolved (libc, socket copy)	9.8 %
`handle_recv`	9.5 %
`handle_sendfile`	8.0 %
`handle_send`	6.5 %
`buffer_reset_and_maybe_shrink`	5.9 %
`produce`	4.7 %
`io_add_send`	4.0 %
`http_response_write_file`	2.8 %
`worker_loop`	2.3 %
`buffer_append`	2.2 %
`io_add_sendfile`	1.3 %
`io_add_recv`	1.1 %

Table 2: `perf record -F 999 -g` on the final `io_uring` server during a 20 s `wrk` load (user-space samples only; kernel symbols are hidden by `kernel.perf_event_paranoid=2`).

Connection stability During long keep-alive stress tests, sporadic `ECONNRESET` errors were observed, consistent with `wrk` tearing down connections mid-flight at the end of a test window. These are handled in the final implementation by closing and cleaning up the connection (returning the slot to the pool) rather than logging an error, since the peer has already left. No errors were observed under normal single-connection `curl` workloads once the keep-alive parsing bug was corrected.

7 Future Work

Recover zero-copy on the body path The $\approx 40\,\%$ throughput gap against `nginx` is almost entirely the memcpy from the mmap'd buffer into the socket buffer that `IORING_OP_SEND` incurs. Two routes worth trying:

- `IORING_OP_SEND_ZC`: we tried this and saw millions of `wrk` read errors under keep-alive, likely because our connection reset on keep-alive raced with the `F_NOTIF` completion. The fix is probably to track

per-connection outstanding ZC notifications and defer slot reuse until the notification has arrived (or to use the recently added `IORING_SEND_ZC_REPORT_USAGE` to detect when ZC degraded to copy).

- **Fix the splice path**: at the cost of a non-trivial state-machine rewrite, the SPLICE pair could work if the per-connection pipe scheme were replaced with a per-worker SP (one pipe per worker, serialising file transfers through a FIFO). This loses parallelism per worker but keeps zero-copy.

Isolate the load generator `wrk` currently shares the 14 cores with the server. Running `wrk` on another machine (or just pinning `wrk` and the workers to disjoint CPU sets) would make the measurement no longer joint-CPU-bound, which is particularly relevant for comparing against `nginx`: `nginx`'s number probably has more headroom than ours.

Registered buffers and fixed files The ring already supports registered buffers (`io_uring_register_buffers`) and registered file descriptors (`io_uring_register_files`). The scaffolding is in `src/io/io_uring.c`; wiring the hot path through `IORING_OP_READ_FIXED`/`IORING_OP_WRITE_FIXED` would remove some per-SQE kernel overhead that currently shows up as the unresolved libc samples in Table 2.

Precomputed response headers Currently `http_response_write_file` builds the response headers at request time via several `buffer_append` calls. Since the files are cached at startup with known sizes and content types, the full response header string could be precomputed once per cached file and stored in `static_file_entry_t`, reducing the per-request header path to a single `memcpy`.

HTTPS Moving from HTTP to HTTPS requires an SSL/TLS certificate. OpenSSL is the standard way of doing it. Adding encryption changes the architecture by introducing a new TLS layer between the HTTP and connection layers. TLS will introduce CPU cost due to encryption and handshake overhead, which makes performance tracking more difficult. This feature is planned after the above optimisations.

8 Conclusion

Non-blocking I/O combined with keep-alive and a multithreaded `SO_REUSEPORT` design brought the `epoll` build to throughput parity with `nginx` on static file serving. The subsequent full `io_uring` port is not a free win: on this kernel, splice is not a usable `sendfile` replacement for a 400-connection pool (pipe-page limits cap per-call splice to 8 KiB), so the body path moved to `IORING_OP_SEND` of an `mmap`'d static cache. That gave up zero-copy and cost $\approx 40\,\%$ of the throughput, but it also cut $p99$ from 40 ms down to 7.6 ms: close to `nginx`, and plausibly a sign that the `epoll` build's tail came from `sendfile`-induced head-of-line blocking inside a worker rather than something fundamentally wrong with the architecture. Recovering zero-copy on `io_uring` (via `SEND_ZC` or a reworked splice) is the next real experiment.

This project really taught me how networking works at a very low level. I also really enjoyed going from a simple kind of 'single-cycle' server, into 'multi-cycle', into a 'superscalar' processor. And as a natural extension, the architecture would become a 'multi-core' server, which is exactly what it did.